Training a Transformer-based Language Model from Scratch

Submitted By:

Vaishnavi Lakade
Nethra Nayak

Subject: Build LLM from Scratch

Sardar Patel Institute of Technology
Department of Computer Engineering

May 10, 2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Large Language Models (LLMs) have transformed the field of Natural Language Processing (NLP) by enabling machines to understand and generate human language effectively. Modern applications such as chatbots, text generation, translation, and summarization are powered by transformer-based architectures. Unlike traditional sequence models such as RNNs and LSTMs, transformers use self-attention mechanisms to capture contextual relationships within text efficiently.

The Transformer architecture forms the foundation of modern language models such as GPT and BERT. It consists of multiple components including embeddings, multi-head self-attention, feed-forward neural networks, residual connections, and normalization layers. These components help the model learn contextual patterns and improve training stability.

In this project, a transformer-based language model is built and trained from scratch using text data obtained from Project Gutenberg. Multiple experiments are conducted by varying learning rates, epochs, transformer layers, and attention heads to analyze their effect on model performance. In addition, ablation studies are performed by removing important transformer components such as normalization layers, residual connections, and feed-forward networks to study their contribution to model learning and convergence.

Chapter 2

Objectives

- To build and train a transformer-based language model from scratch.
- To preprocess textual data obtained from Project Gutenberg for language model training.
- To train and evaluate a baseline transformer model using training and validation loss metrics.
- To study the effect of different learning rates and epochs on model convergence and performance.
- To analyze the impact of varying the number of transformer layers.
- To evaluate the influence of different attention head configurations on model learning.
- To perform ablation studies by removing normalization layers, residual connections, and feed-forward networks.
- To compare and analyze the performance of different model configurations using graphical visualizations and loss metrics.

Chapter 3

Dataset Description

3.1 Dataset Source

The dataset used in this project was obtained from Project Gutenberg, which contains a large collection of publicly available literary texts. Multiple book texts were combined into a single dataset for training the transformer-based language model. The dataset consists of natural language sentences, dialogues, and narrative text suitable for next-token prediction tasks.

3.2 Data Preprocessing

Before training the model, the raw text data was cleaned and normalized using several preprocessing steps to improve consistency and reduce noise in the dataset.

- All text was converted into lowercase to maintain uniformity between tokens.
- Underscore characters and unnecessary symbols present in the original Gutenberg text were removed.
- Regular expressions were used to retain only letters, numbers, spaces, and basic punctuation symbols such as periods, commas, question marks, exclamation marks, semicolons, and colons.
- Multiple spaces and newline characters were replaced with single spaces to create a cleaner text format.

The preprocessing helped in reducing vocabulary redundancy and improving token consistency during training.

3.3 Tokenization

After preprocessing, the cleaned text was tokenized into smaller units called tokens. A regular expression based tokenizer was used to split the text into words and punctuation symbols separately.

The tokenizer extracted:

- words,
- numbers,
- punctuation symbols.

This tokenized representation formed the input for vocabulary generation and model training.

3.4 Vocabulary Generation

A vocabulary was created using all unique tokens present in the dataset. The vocabulary was sorted and each token was assigned a unique integer index.

Two dictionaries were created:

- `word_to_idx`: Maps each token to a numerical index.
- `idx_to_word`: Maps numerical indices back to their corresponding tokens.

The total number of unique tokens determined the final vocabulary size used by the model.

3.5 Token Encoding

Once the vocabulary was created, all tokens in the dataset were converted into their corresponding numerical IDs using the `word_to_idx` mapping. This encoded representation allowed the textual data to be processed numerically by the transformer model.

3.6 Sequence Generation

To train the language model, fixed-length input-target sequences were generated from the encoded text data.

A sequence length of 32 tokens was used. For each training sample:

- The input sequence consisted of 32 consecutive token IDs.

- The target sequence contained the next token for each position in the input sequence.

This approach enabled the model to learn next-token prediction by predicting the subsequent token based on previous contextual information.

Finally, the generated input and target sequences were converted into PyTorch tensors for efficient model training.

Chapter 4

Methodology

4.1 Train-Validation Split

The generated input-target sequences were divided into training and validation datasets using an 80:20 ratio. The training dataset was used for model learning, while the validation dataset was used to evaluate model performance during training.

4.2 Dataset and DataLoader

A custom PyTorch dataset class was created to efficiently handle the input and target sequences. PyTorch DataLoaders were used to load the data in batches during training and validation. A batch size of 32 was used for model training.

4.3 Model Architecture

A GPT-style decoder-only transformer architecture was implemented from scratch using PyTorch. The model consists of token embeddings, positional embeddings, multi-head self-attention layers, feed-forward neural networks, residual connections, normalization layers, and dropout regularization. Multiple transformer blocks were stacked sequentially to form the complete language model.

4.4 Embedding Layer

The embedding layer transforms input token IDs into dense vector representations that can be processed by the transformer model. Two types of embeddings were used:

- **Token Embeddings** — represent the semantic meaning of tokens.

- **Positional Embeddings** — encode the positional information of tokens within a sequence.

The final input representation was obtained by adding token embeddings and positional embeddings. This enables the model to understand both the meaning of words and their order in the sequence.

4.5 Multi-Head Self-Attention

Multi-head self-attention was used to capture contextual relationships between tokens in a sequence. Query (Q), Key (K), and Value (V) matrices were generated using linear transformations of the input embeddings.

Scaled dot-product attention was then applied to compute attention weights between tokens, allowing the model to focus on relevant contextual information.

A causal attention mask was used during training to prevent the model from accessing future tokens, enabling autoregressive next-token prediction.

4.6 Feed Forward Network

Each transformer block contains a Feed Forward Neural Network (FFN) consisting of two linear layers with GELU activation. The FFN performs nonlinear transformations on the attention outputs, helping the model learn richer and more complex feature representations.

4.7 Residual Connections and Layer Normalization

Residual shortcut connections were applied around both the attention and feed-forward blocks to improve gradient flow and stabilize deep network training. These connections help preserve information from earlier layers and reduce optimization difficulties.

Layer normalization was applied before attention and feed-forward operations to stabilize activation distributions and improve convergence during training.

4.8 Loss Function and Optimizer

Cross-Entropy Loss was used as the objective function for next-token prediction. The Adam optimizer was used for model optimization with a learning rate of 1×10^{-4} due to its efficient adaptive gradient updates and stable convergence behavior.

4.9 Training Procedure

During training, batches of input sequences were passed through the transformer model to predict the next token at each position. Training and validation loss values were recorded across epochs to analyze model convergence and performance under different experimental configurations.

4.10 Model Configuration

The baseline transformer model was configured using the following parameters:

Parameter	**Value**
Embedding Dimension	128
Number of Attention Heads	4
Number of Transformer Layers	3
Context Length	32
Dropout Rate	0.2
Batch Size	32
Learning Rate	1×10^{-4}

Table 4.1: Baseline GPT Model Configuration

Chapter 5

Baseline Model Training

5.1 Training Function

A custom training function was implemented to train and evaluate the transformer model across multiple epochs. During each epoch, the model was trained using the training dataset and evaluated using the validation dataset.

The training process included:

- Forward propagation through the transformer model
- Cross-entropy loss computation for next-token prediction
- Backpropagation using gradient descent
- Parameter optimization using the Adam optimizer

Average training loss and validation loss were computed after each epoch and stored for performance analysis.

5.2 Baseline Model Training

The baseline transformer model was trained for 10 epochs using the following configuration:

Parameter	**Value**
Embedding Dimension	128
Transformer Layers	3
Attention Heads	4
Learning Rate	1×10^{-4}
Dropout Rate	0.2
Epochs	10

Table 5.1: Baseline Training Configuration

Training and validation loss values were recorded across epochs to evaluate model convergence and generalization performance.

5.3 Baseline Results

The training and validation loss curves obtained from baseline training are shown below.

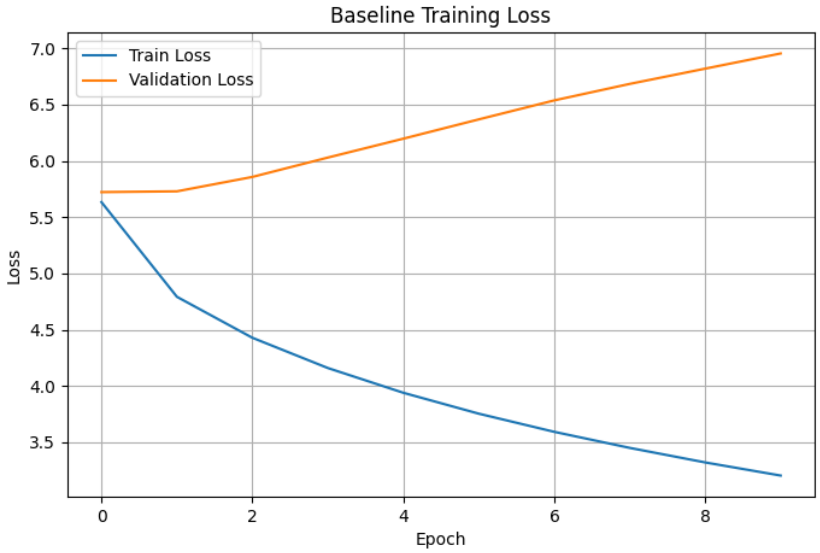

Figure 5.1: Baseline Training and Validation Loss

5.4 Analysis

The training loss decreased steadily across epochs, indicating that the model successfully learned token-level patterns from the dataset. The validation loss also decreased initially but later began diverging slightly from the training loss, indicating the onset of overfitting.

The observed overfitting is expected because the dataset size is relatively limited compared to the capacity of the transformer model. Despite this, the model was able to learn meaningful contextual patterns and generate domain-relevant text related to the training dataset.

The baseline model serves as the reference configuration for all subsequent experiments involving learning rates, transformer layers, attention heads, and ablation studies.

Chapter 6

Experiment 1: Epochs and Learning Rate

6.1 Objective

In this experiment, the baseline transformer architecture was kept fixed while varying the learning rate and number of training epochs. The objective was to analyze model convergence, optimization stability, and overfitting behavior under different training conditions.

The baseline architecture used:

- Embedding Dimension = 128
- Transformer Layers = 3
- Attention Heads = 4

6.2 Experimental Configurations

The following learning rates and epoch values were tested:

Experiment	**Learning Rate**	**Epochs**
Experiment A	1×10^{-4}	5
Experiment B	1×10^{-4}	10
Experiment C	5×10^{-4}	5
Experiment D	1×10^{-3}	5
Experiment E	5×10^{-5}	5

Table 6.1: Experiment 1 Configurations

6.3 Validation Loss Comparison

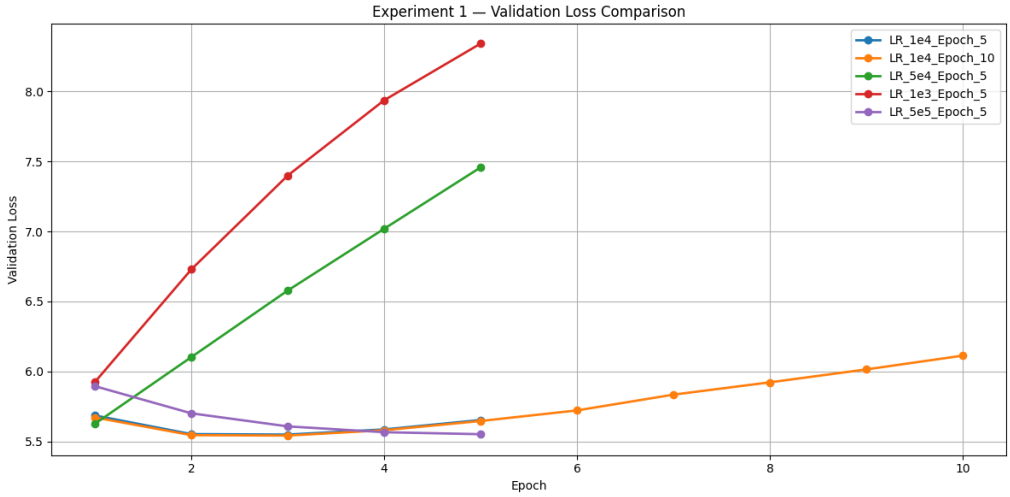

Figure 6.1: Validation Loss Comparison for Different Learning Rates and Epochs

6.4 Training Loss Comparison

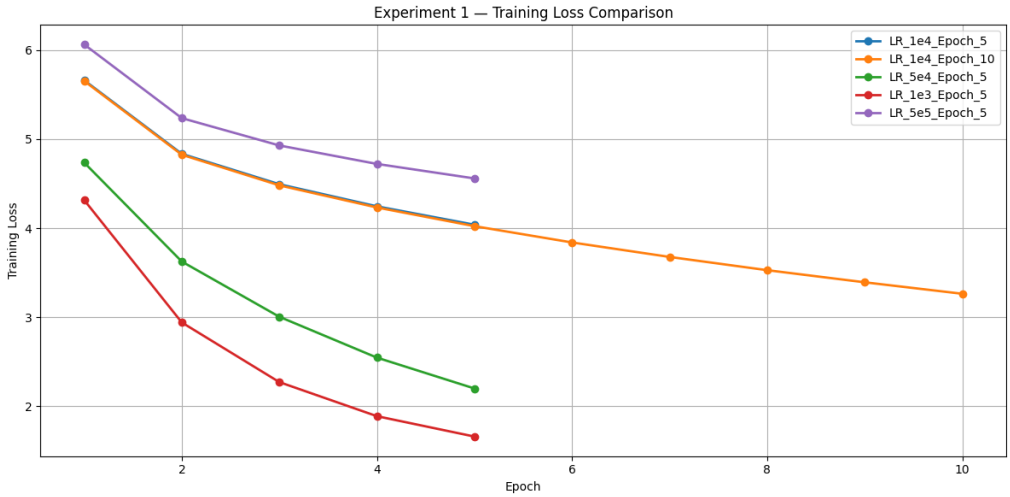

Figure 6.2: Training Loss Comparison for Different Learning Rates and Epochs

6.5 Analysis

6.5.1 Learning Rate Observations

- **Learning Rate** $= 5 \times 10^{-5}$**:** The model showed very slow convergence. Training loss decreased gradually, but validation loss remained relatively high, indicating insufficient learning within the given epochs.

- **Learning Rate** $= 1 \times 10^{-4}$**:** This configuration produced the most stable training behavior. Training loss decreased consistently with moderate overfitting after several epochs.
- **Learning Rate** $= 5 \times 10^{-4}$**:** The model converged faster initially, but validation loss diverged earlier, suggesting unstable generalization.
- **Learning Rate** $= 1 \times 10^{-3}$**:** Training became unstable with fluctuating loss values. The optimizer overshot minima frequently, resulting in poor validation performance.

6.5.2 Epoch Observations

- Increasing training from 5 to 10 epochs with learning rate 1×10^{-4} reduced training loss further but increased validation loss, indicating overfitting.
- Longer training on the relatively small dataset caused the model to memorize training sequences instead of improving generalization.

6.6 Inference

The results show that learning rate and number of epochs significantly influence transformer training behavior.

Among all configurations, a learning rate of 1×10^{-4} with 5 epochs provided the best balance between convergence and generalization. Higher learning rates enabled faster optimization but resulted in unstable training and poorer validation performance.

The experiment also demonstrated that increasing epochs beyond a certain point does not necessarily improve model performance, especially when training on smaller datasets. Early stopping and stronger regularization techniques would be required for longer training durations.

Chapter 7

Experiment 2: Transformer Layers

7.1 Objective

In this experiment, the number of transformer layers was varied while keeping all other hyperparameters fixed. The objective was to analyze the effect of architectural depth on representation learning, convergence behavior, and generalization performance.

The following hyperparameters were kept constant:

- Embedding Dimension = 128
- Attention Heads = 4
- Learning Rate = 1×10^{-4}
- Epochs = 5

7.2 Experimental Configurations

The model was trained using different numbers of transformer layers.

Experiment	**Transformer Layers**
Experiment A	1
Experiment B	3
Experiment C	5
Experiment D	7
Experiment E	12

Table 7.1: Experiment 2 Configurations

7.3 Validation Loss Comparison

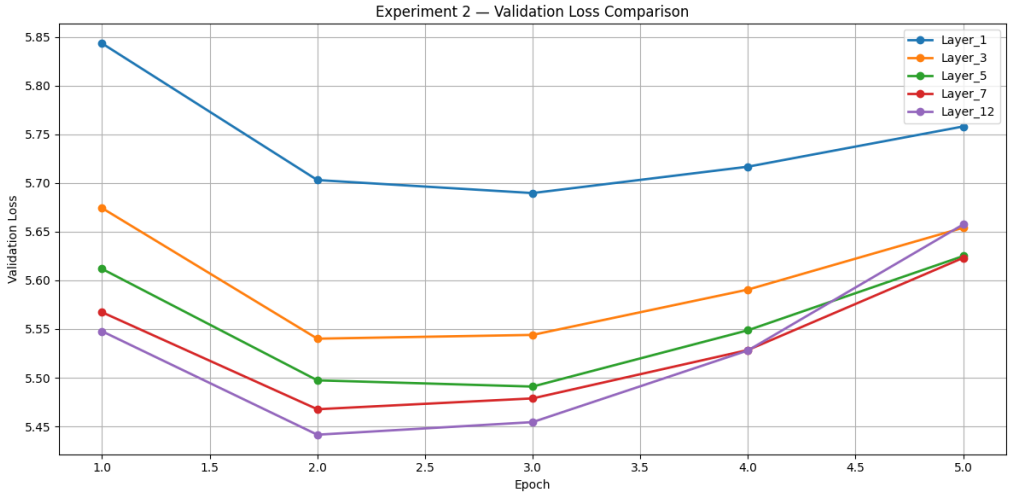

Figure 7.1: Validation Loss Comparison for Different Transformer Layers

7.4 Training Loss Comparison

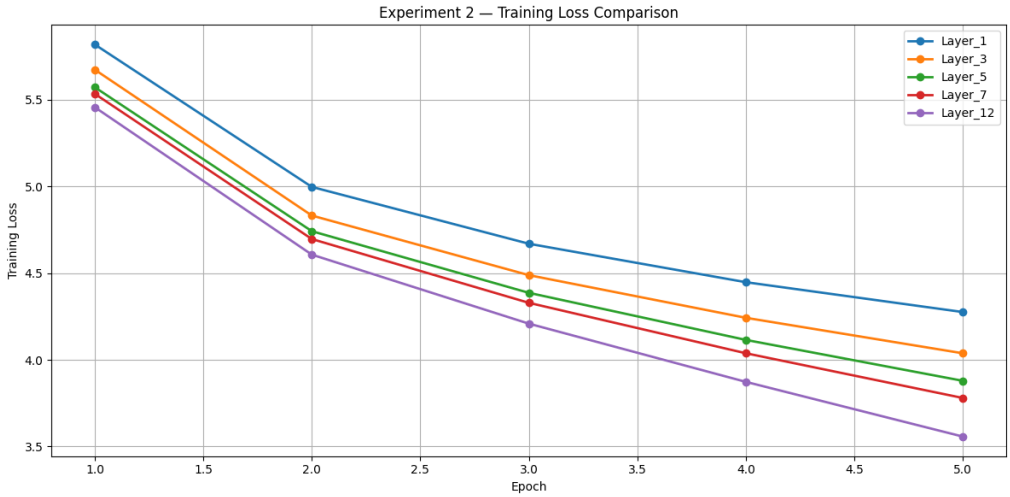

Figure 7.2: Training Loss Comparison for Different Transformer Layers

7.5 Analysis

- **1 Layer:** The model showed the highest training and validation loss. The shallow architecture lacked sufficient capacity to capture meaningful contextual relationships in the text.
- **3 Layers:** This configuration provided a balanced tradeoff between learning capacity and generalization. Training converged smoothly with moderate validation

loss.

- **5 Layers:** The model achieved lower training loss initially, but validation loss increased more rapidly, indicating the beginning of overfitting.
- **7 Layers:** Training loss decreased faster than the shallower models, but validation performance worsened significantly. The model began memorizing training sequences instead of learning generalizable patterns.
- **12 Layers:** This configuration showed the most severe overfitting. Although training loss was the lowest, validation loss remained highest due to excessive model complexity for the relatively small dataset.

7.6 Inference

The experiment demonstrates that increasing transformer depth improves learning capacity only up to a certain point. Shallow models tend to underfit due to limited representational power, whereas excessively deep models overfit aggressively on smaller datasets.

For this dataset, transformer models with 3 to 5 layers provided the best balance between convergence and generalization. Increasing architectural depth beyond this range increased model complexity without improving validation performance.

The results highlight the importance of selecting model depth according to dataset size and available training data.

Chapter 8

Experiment 3: Attention Heads

8.1 Objective

In this experiment, the number of attention heads was varied while keeping all other hyperparameters fixed. The objective was to analyze how multi-head attention affects representation learning, contextual understanding, and model generalization.

The following hyperparameters were kept constant:

- Embedding Dimension = 128
- Transformer Layers = 3
- Learning Rate = 1×10^{-4}
- Epochs = 5

8.2 Experimental Configurations

The model was trained using different numbers of attention heads.

Experiment	**Attention Heads**
Experiment A	1
Experiment B	2
Experiment C	4
Experiment D	8

Table 8.1: Experiment 3 Configurations

8.3 Validation Loss Comparison

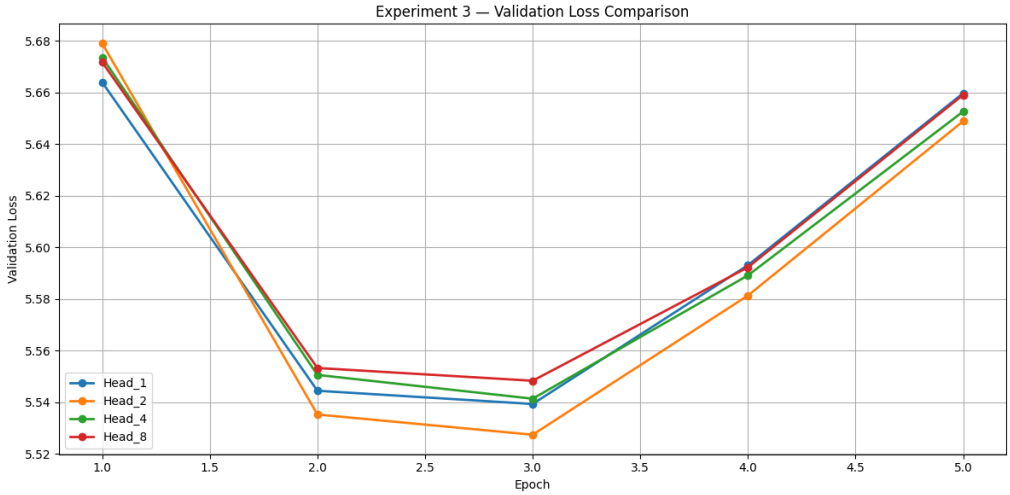

Figure 8.1: Validation Loss Comparison for Different Attention Heads

8.4 Training Loss Comparison

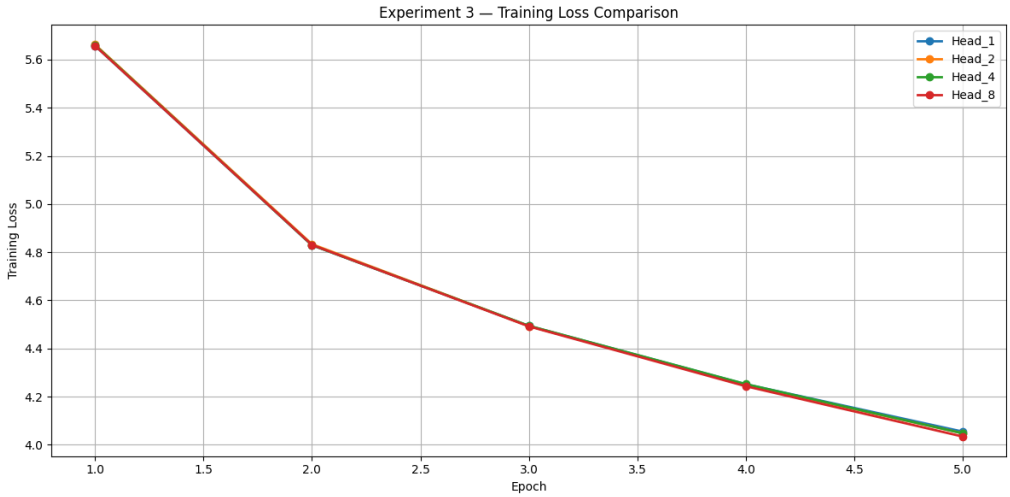

Figure 8.2: Training Loss Comparison for Different Attention Heads

8.5 Analysis

- **1 Head:** The model showed slower convergence and higher validation loss. A single attention head could capture only limited contextual relationships within the text.
- **2 Heads:** The model showed slight improvement over the single-head configuration. Multiple heads allowed the model to capture somewhat richer contextual dependencies.

- **4 Heads:** This configuration achieved the best balance between learning capacity and generalization. The model effectively captured multiple contextual patterns simultaneously.
- **8 Heads:** Increasing the number of heads further produced diminishing returns. Since the embedding dimension remained fixed at 128, each head operated on fewer dimensions, reducing representational capacity per head.

8.6 Inference

The experiment demonstrates that multi-head attention improves transformer performance by allowing the model to learn different contextual relationships in parallel.

However, increasing the number of attention heads beyond an optimal value does not necessarily improve performance. With a fixed embedding dimension, too many heads reduce the dimensional capacity available to each head, limiting their effectiveness.

For the given architecture and dataset, 4 attention heads provided the best generalization performance and training stability. The results highlight the importance of balancing embedding dimension and attention head count when designing transformer architectures.

Chapter 9

Experiment 4: Ablation Studies

9.1 Objective

In this experiment, key transformer components were selectively removed to analyze their impact on optimization stability, convergence behavior, and representation learning.

The following ablation studies were performed:

- Removal of Layer Normalization
- Removal of Residual Connections
- Removal of Feed Forward Neural Network (FFN)

All remaining hyperparameters were kept fixed to isolate the contribution of each architectural component.

The baseline configuration used:

- Embedding Dimension = 128
- Transformer Layers = 3
- Attention Heads = 4
- Learning Rate = 1×10^{-4}
- Epochs = 5

9.2 Ablation A: Removing Layer Normalization

In this ablation study, all Layer Normalization operations were removed from the transformer architecture while preserving attention, feed-forward, and residual pathways.

9.2.1 Validation Loss

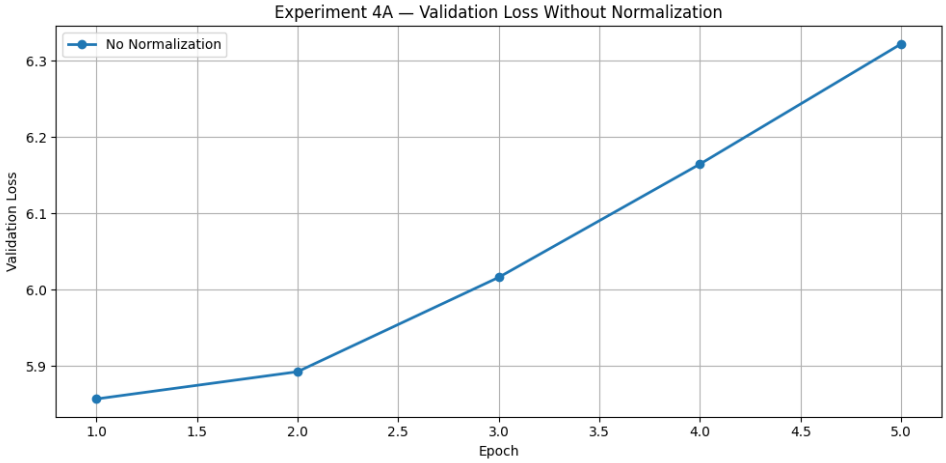

Figure 9.1: Validation Loss Without Layer Normalization

9.2.2 Training Loss

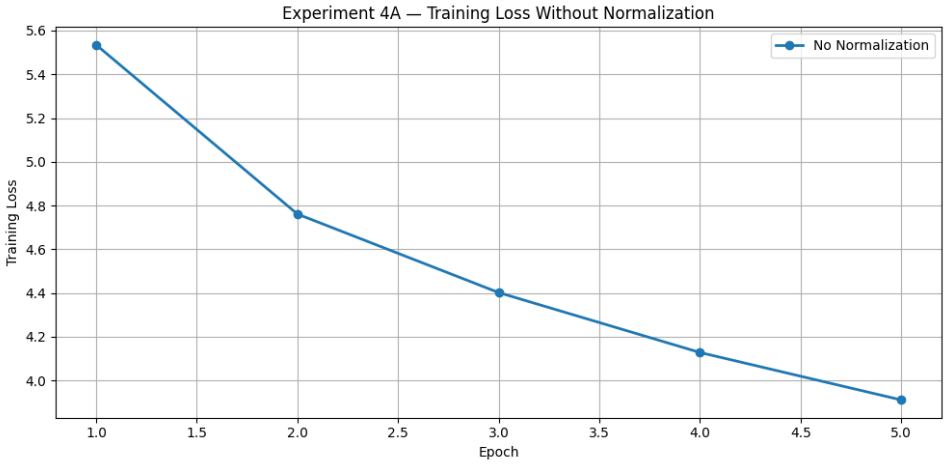

Figure 9.2: Training Loss Without Layer Normalization

9.2.3 Analysis

- Training loss decreased more slowly and less consistently compared to the baseline model.
- Validation loss increased rapidly after early epochs, indicating unstable learning and poor generalization.
- The train-validation gap became significantly larger, suggesting faster overfitting.

9.2.4 Inference

Layer Normalization helps stabilize activation distributions across transformer layers and improves optimization stability. Without normalization, gradients become inconsistent across layers, making training less stable and reducing generalization capability.

The generated text without normalization was more repetitive and less coherent compared to the baseline model.

9.3 Ablation B: Removing Residual Connections

In this ablation study, all residual shortcut connections were removed while preserving attention and feed-forward operations.

9.3.1 Validation Loss

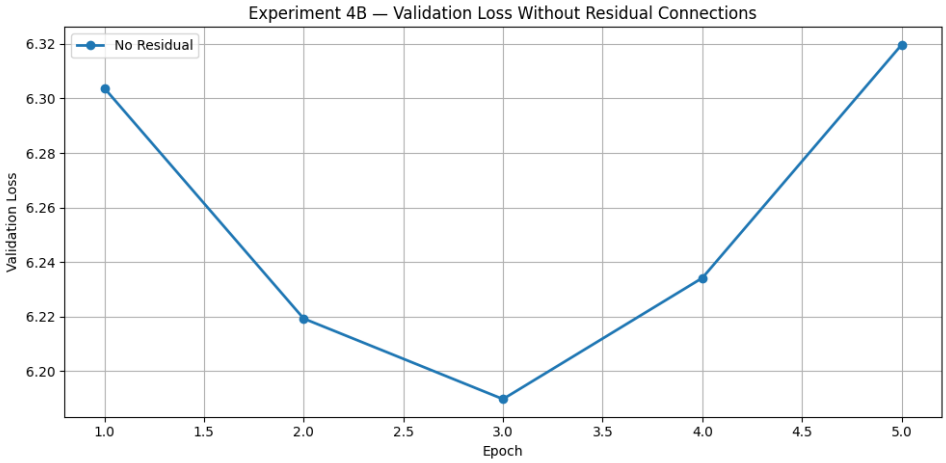

Figure 9.3: Validation Loss Without Residual Connections

9.3.2 Training Loss

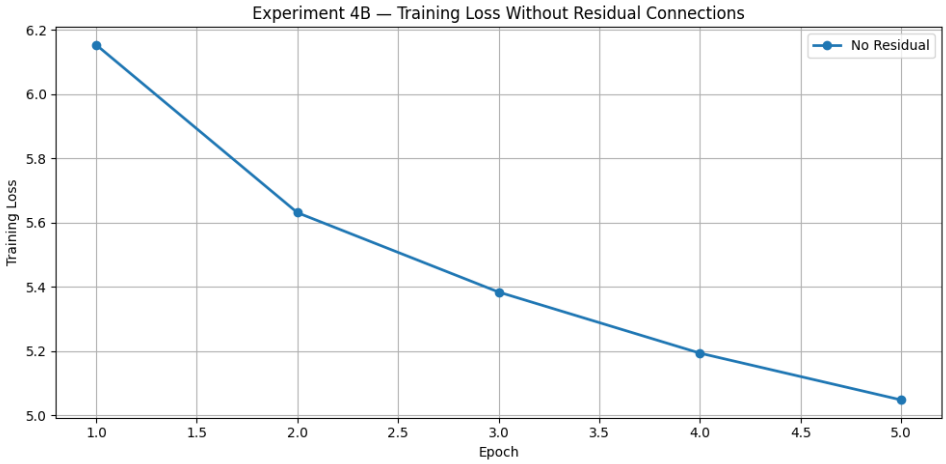

Figure 9.4: Training Loss Without Residual Connections

9.3.3 Analysis

- Removing residual connections caused significant degradation in convergence stability.
- Both training and validation loss remained noticeably higher than the baseline configuration.
- The model struggled to improve effectively across epochs due to poor gradient propagation.

9.3.4 Inference

Residual connections are critical for stable deep learning optimization because they allow gradients to flow directly through the network. Without shortcut pathways, deeper layers become difficult to optimize, leading to slower convergence and poorer representation learning.

Generated text without residual connections became grammatically inconsistent and contextually incoherent.

9.4 Ablation C: Removing Feed Forward Network

In this ablation study, the Feed Forward Neural Network (FFN) was removed from all transformer layers while preserving attention and residual pathways.

9.4.1 Validation Loss

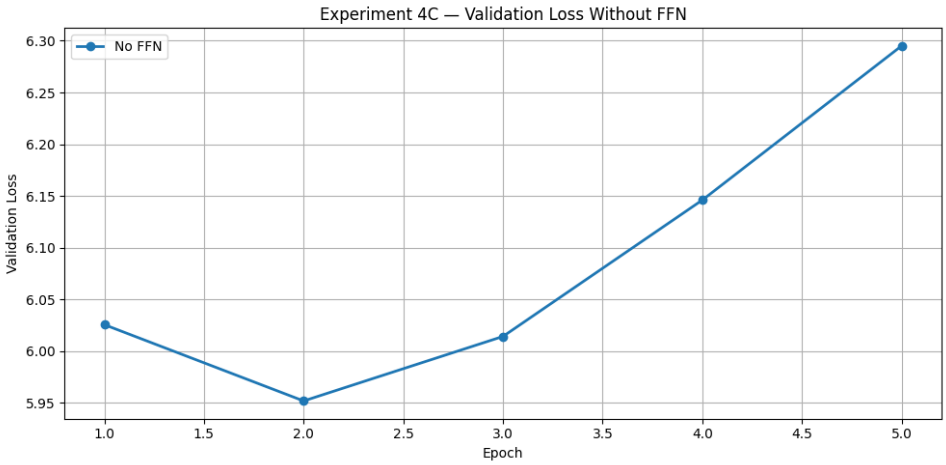

Figure 9.5: Validation Loss Without Feed Forward Network

9.4.2 Training Loss

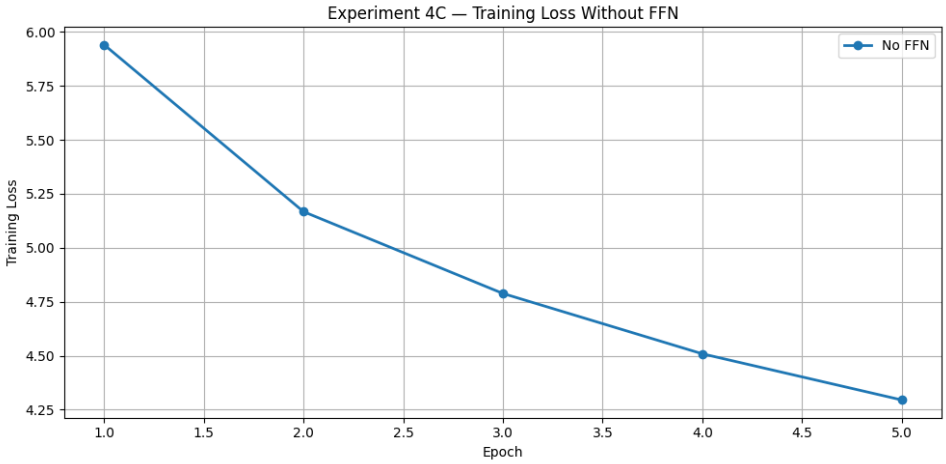

Figure 9.6: Training Loss Without Feed Forward Network

9.4.3 Analysis

- The model continued learning because the attention mechanism was still present.
- However, both training and validation loss remained higher than the baseline model.
- The rate of convergence was slower and final performance was reduced.

9.4.4 Inference

The Feed Forward Network introduces nonlinear feature transformation after the attention operation. While attention mixes contextual information across positions, the FFN

refines these representations through nonlinear transformations.

Without FFN layers, the transformer loses a significant portion of its representational capacity, resulting in weaker contextual understanding and repetitive text generation.

9.5 Overall Ablation Summary

Component Removed	**Impact**	**Severity**
Layer Normalization	Unstable optimization and poor generalization	High
Residual Connections	Slow convergence and weak gradient flow	High
Feed Forward Network	Reduced representational capacity	Medium

Table 9.1: Overall Ablation Study Summary

The ablation studies demonstrate that all major transformer components contribute significantly to model performance. Residual connections and Layer Normalization primarily improve optimization stability, while the Feed Forward Network improves representation learning and contextual understanding.

Chapter 10

Comparative Analysis

10.1 Learning Rate and Epoch Comparison

Configuration	Training Behavior	Validation Behavior	Observation
LR = 5×10^{-5}, 5 Epochs	Slow convergence	Validation loss remained high	Model learned too slowly within limited epochs
LR = 1×10^{-4}, 5 Epochs	Stable convergence	Best validation performance	Best balance between convergence and generalization
LR = 1×10^{-4}, 10 Epochs	Lower training loss	Validation loss increased after few epochs	Overfitting observed after extended training
LR = 5×10^{-4}, 5 Epochs	Fast initial convergence	Validation loss diverged earlier	Learning rate slightly too aggressive
LR = 1×10^{-3}, 5 Epochs	Unstable optimization	Highest validation fluctuations	Optimizer overshot minima frequently

Table 10.1: Comparison of Learning Rate and Epoch Configurations

10.2 Transformer Layer Comparison

Layers	Training Performance	Validation Performance	Observation
1 Layer	Highest training loss	Highest validation loss	Insufficient depth to capture contextual dependencies
3 Layers	Stable convergence	Moderate validation loss	Good balance between capacity and generalization
5 Layers	Lower training loss	Validation loss increased faster	Additional depth increased overfitting
7 Layers	Fast training convergence	Poor validation performance	Model memorized training sequences
12 Layers	Lowest training loss	Highest validation loss	Severe overfitting due to excessive model complexity

Table 10.2: Comparison of Transformer Layer Configurations

10.3 Attention Head Comparison

Attention Heads	Training Behavior	Validation Behavior	Observation
1 Head	Slow convergence	Higher validation loss	Unable to capture multiple contextual patterns
2 Heads	Improved learning	Slight validation improvement	Better contextual representation
4 Heads	Stable convergence	Best validation performance	Best balance between attention diversity and head capacity
8 Heads	Comparable training loss	No major validation improvement	Reduced per-head dimensional capacity

Table 10.3: Comparison of Attention Head Configurations

10.4 Ablation Study Comparison

Removed Component	Observed Impact	Inference
Layer Normalization	Unstable training and poor generalization	Normalization stabilizes activations and improves optimization
Residual Connections	Slow convergence and poor gradient flow	Residual paths are critical for stable deep learning
Feed Forward Network	Higher loss and weaker contextual learning	FFN improves nonlinear representation learning

Table 10.4: Comparison of Ablation Study Results

Chapter 11

Conclusion and Future Work

11.1 Conclusion

In this project, a transformer-based language model was implemented and trained from scratch using textual data obtained from Project Gutenberg. The project explored multiple aspects of transformer training, including optimization behavior, architectural depth, attention mechanisms, and the contribution of key transformer components.

The baseline model demonstrated successful learning of contextual language patterns through steady reduction in training loss and meaningful text generation. Multiple experiments were then conducted by varying learning rates, training epochs, transformer layers, and attention heads.

The results showed that model performance is highly dependent on selecting appropriate hyperparameters and architectural configurations. A learning rate of 1×10^{-4} with 5 epochs provided the best balance between convergence and generalization for the given dataset. Increasing transformer depth improved learning capacity up to a certain point, after which deeper models overfit due to limited training data. Similarly, multi-head attention improved contextual learning, although excessive attention heads reduced representational efficiency.

The ablation studies further demonstrated the importance of Layer Normalization, residual connections, and Feed Forward Networks in transformer architectures. Removing these components resulted in unstable optimization, weaker gradient propagation, and reduced representational capacity.

Overall, the project provided practical understanding of transformer architectures, training dynamics, and the role of major architectural components in Large Language Models.

11.2 Future Work

Several improvements can be explored in future work to further enhance model performance and scalability:

- Training on larger and more diverse text datasets to improve generalization capability.
- Using subword tokenization techniques such as Byte Pair Encoding (BPE) instead of word-level tokenization.
- Increasing context length and embedding dimensions for richer contextual learning.
- Applying regularization techniques such as early stopping and learning rate scheduling to reduce overfitting.
- Experimenting with larger transformer architectures and pretrained initialization methods.
- Evaluating generated text quality using additional NLP evaluation metrics.

Bibliography

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, *Attention Is All You Need*, Advances in Neural Information Processing Systems (NeurIPS), 2017.

[2] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever, *Improving Language Understanding by Generative Pre-Training*, OpenAI, 2018.

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, NAACL, 2019.

Appendix A

Code Repository

The complete implementation code, training scripts, experiment configurations, and generated results for this project are available on GitHub.

GitHub Repository Link:

`https://github.com/nethra815/LLM_Assignment.git`